

**LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM**

Laparak Minggu 14

2311102280_Loisa Vanica Saragih

KELAS IF-11-03

Dosen: Cahyo Prihantoro S.Kom., M.Eng.



**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM
PURWOKERTO
2026**

Link github : <https://github.com/Loil60/Webpro>

Source Code Database character_model:

```
class Character {
  final int id;
  final String name;
  final String username;

  Character({required this.id, required this.name, required
this.username});

  factory Character.fromMap(Map<String, dynamic> map) {
    return Character(
      id: map['id'],
      name: map['name'],
      username: map['username'] ?? 'Unknown Username',
    );
  }

  Map<String, dynamic> toMap() {
    return {'id': id, 'name': name, 'username': username};
  }
}
```

Source Code Database db_helper:

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
import 'character_model.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('characters.db');
    return _database!;
  }
}
```

```

Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    return await openDatabase(path, version: 1, onCreate: _createDB);
}

Future _createDB(Database db, int version) async {
    await db.execute('''
        CREATE TABLE characters (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL,
            username TEXT NOT NULL
        )
    ''');
}

Future<void> insertCharacter(Character character) async {
    final db = await instance.database;
    await db.insert(
        'characters',
        character.toMap(),
        conflictAlgorithm: ConflictAlgorithm.replace,
    );
}

Future<List<Character>> getAllCharacters() async {
    final db = await instance.database;
    final maps = await db.query('characters');

    return maps.map((map) => Character.fromMap(map)).toList();
}

Future<void> clearTable() async {
    final db = await instance.database;
    await db.delete('characters');
}
}

```

Source Code Database main:

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'character_model.dart';
import 'db_helper.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'SQLite & HTTP Demo',
      theme: ThemeData.dark(useMaterial3: true),
      home: const CharacterListScreen(),
    );
  }
}

class CharacterListScreen extends StatefulWidget {
  const CharacterListScreen({super.key});

  @override
  State<CharacterListScreen> createState() =>
    _CharacterListScreenState();
}

class _CharacterListScreenState extends State<CharacterListScreen> {
  List<Character> _characters = [];
  bool _isLoading = false;

  @override
  void initState() {
    super.initState();
    _loadFromLocal();
  }
}
```

```

Future<void> _loadFromLocal() async {
  setState(() => _isLoading = true);
  final localData = await
DatabaseHelper.instance.getAllCharacters();

  setState(() {
    _characters = localData;
    _isLoading = false;
  });
}

// Fungsi Menghapus Semua Data dari SQLite
Future<void> _deleteAllData() async {
  bool? confirm = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Hapus Semua Data?'),
      content: const Text('apakah anda yakin?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text('Batal'),
        ),
        TextButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text('Hapus', style: TextStyle(color:
Colors.red)),
        ),
      ],
    ),
  );

  if (confirm != true) return;

  setState(() => _isLoading = true);

  try {
    await DatabaseHelper.instance.clearTable();

    setState(() {
      _characters.clear();
    });
  }
}

```

```

        if (mounted) {
            ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(
                    content: Text('Semua data berhasil dihapus dari
sistem.'),
                ),
            );
        }
    } catch (e) {
        if (mounted) {
            ScaffoldMessenger.of(
                context,
            ).showSnackBar(SnackBar(content: Text('Terjadi kesalahan:
$e')));
        }
    } finally {
        setState(() => _isLoading = false);
    }
}

Future<void> _fetchFromAPI() async {
    setState(() => _isLoading = true);

    try {
        final response = await http.get(
            Uri.parse('https://jsonplaceholder.typicode.com/users'),
        );

        if (response.statusCode == 200) {
            final List<dynamic> dataJson = json.decode(response.body);

            await DatabaseHelper.instance.clearTable();

            List<Character> freshCharacters = [];

            for (var item in dataJson) {
                final char = Character.fromMap(item);
                freshCharacters.add(char);
                await DatabaseHelper.instance.insertCharacter(char);
            }

            setState(() => _characters = freshCharacters);

```

```

        if (mounted) {
            ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('Berhasil sync data dari
server!'))),
        );
    }
}
} catch (e) {
    if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(
                content: Text('Gagal konek internet, pastikan online.'),
            ),
        );
    }
} finally {
    setState(() => _isLoading = false);
}
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            title: const Text('Ensiklopedia NPC'),
            actions: [
                IconButton(
                    icon: const Icon(Icons.delete_forever, color:
Colors.redAccent),
                    tooltip: 'Hapus Semua Data',
                    onPressed: _characters.isEmpty ? null : _deleteAllData,
                ),
                IconButton(
                    icon: const Icon(Icons.cloud_download),
                    tooltip: 'Sync dari Server',
                    onPressed: _fetchFromAPI,
                ),
            ],
        ),
        body: _isLoading
            ? const Center(child: CircularProgressIndicator())

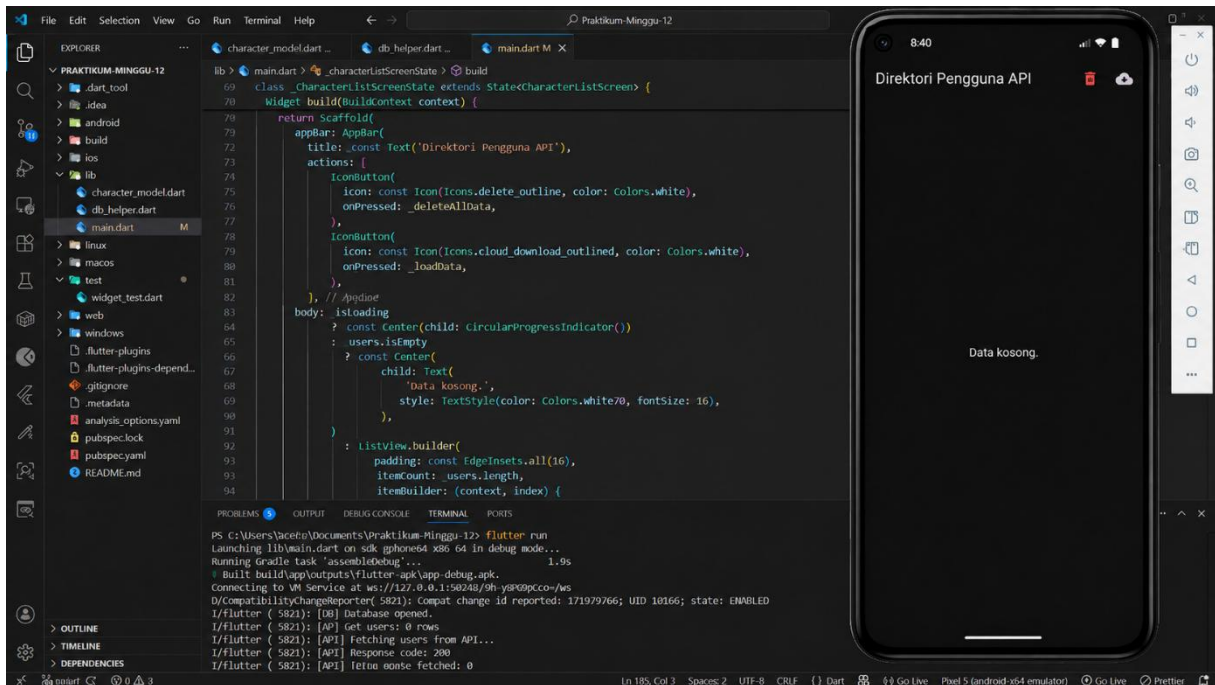
```

```

: _characters.isEmpty
? const Center(child: Text('Data kosong.'))
: ListView.builder(
  itemCount: _characters.length,
  itemBuilder: (context, index) {
    final char = _characters[index];
    return ListTile(
      leading: CircleAvatar(
        backgroundColor: Colors.deepPurple,
        child: Text(char.id.toString()),
      ),
      title: Text(char.name),
      subtitle: Text('Username: ${char.username}'),
    );
  },
),
);
}
}

```

ScreenShot Output:



Deskripsi:

Gambar di atas menunjukkan tampilan hasil aplikasi Flutter bernama Direktori Pengguna API yang digunakan untuk menampilkan data pengguna dari server. Pada kondisi awal, aplikasi menampilkan pesan "Data kosong." karena data belum berhasil dimuat atau belum tersimpan di database lokal. Aplikasi ini menggunakan konsep pengambilan data melalui HTTP request dari REST API, kemudian data tersebut disimpan ke dalam database lokal menggunakan SQLite. Dengan adanya penyimpanan lokal, data yang sudah diperoleh dapat tetap ditampilkan kembali tanpa harus selalu mengambil ulang dari server.

Setelah proses sinkronisasi dilakukan, aplikasi berhasil menampilkan daftar data pengguna dalam bentuk list. Setiap data ditampilkan dengan nomor urut, nama pengguna, dan username. Tombol unduh atau sinkronisasi digunakan untuk mengambil data dari API, sedangkan tombol hapus digunakan untuk menghapus data yang tersimpan pada database lokal. Tampilan ini membuktikan bahwa aplikasi telah berhasil menghubungkan proses pengambilan data dari server dengan penyimpanan lokal SQLite serta menampilkannya melalui antarmuka Flutter.

Source Code counter_provider

```
import 'package:flutter/material.dart';

class CounterProvider with ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void increment() {
    _counter++;
    notifyListeners();
  }

  void decrement() {
    if (_counter > 0) {
      _counter--;
      notifyListeners();
    }
  }
}
```

Source Code counter_screen

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'counter_provider.dart';

class CounterScreen extends StatelessWidget {
  const CounterScreen({super.key});

  @override
  Widget build(BuildContext context) {
```

```

return Scaffold(
  appBar: AppBar(
    title: const Text("Ini adalah aplikasi Counter"),
    backgroundColor: Colors.red,
  ),
  body: Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        const Text("Isi Counter: "),
        Consumer<CounterProvider>(
          builder: (context, counter, child) {
            return Text(
              "${counter.counter}",
              style: const TextStyle(fontSize: 40, fontWeight:
FontWeight.bold),
            );
          },
        ),
      ],
    ),
  ),
  floatingActionButton: Column(
    mainAxisAlignment: MainAxisAlignment.end,
    children: [
      FloatingActionButton(
        heroTag: 'btn1',
        onPressed: () {
          Provider.of<CounterProvider>(context, listen:
false).increment();
        },
        child: const Icon(Icons.add),
      ),
      const SizedBox(height: 10),
      FloatingActionButton(
        heroTag: 'btn2',
        onPressed: () {
          Provider.of<CounterProvider>(context, listen:
false).decrement();
        },
        child: const Icon(Icons.remove),
      ),
    ],
  ),
);

```

```

        ],
      ),
    );
  }
}

```

Source Code main

```

import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'package:provider/provider.dart';

// Import disesuaikan dengan folder di screenshot VS Code kamu
import 'character_model.dart';
import 'db_helper.dart';
import 'provider/counter_provider.dart';
import 'provider/counter_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CounterProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Gabungan SQLite & Provider',
      theme: ThemeData.dark(useMaterial3: true),
      home: const CharacterListScreen(),
    );
  }
}

class CharacterListScreen extends StatefulWidget {

```

```

const CharacterListScreen({super.key});

@override
State<CharacterListScreen> createState() =>
  _CharacterListScreenState();
}

class _CharacterListScreenState extends State<CharacterListScreen> {
  List<Character> _characters = [];
  bool _isLoading = false;

  @override
  void initState() {
    super.initState();
    _loadFromLocal();
  }

  Future<void> _loadFromLocal() async {
    setState(() => _isLoading = true);
    final localData = await
DatabaseHelper.instance.getAllCharacters();
    setState(() {
      _characters = localData;
      _isLoading = false;
    });
  }

  Future<void> _deleteAllData() async {
    bool? confirm = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Hapus Semua Data?'),
        content: const Text('apakah anda yakin?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Batal'),
          ),
          TextButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Hapus', style: TextStyle(color:
Colors.red)),

```

```

        ),
    ],
),
);

if (confirm != true) return;

setState(() => _isLoading = true);

try {
    await DatabaseHelper.instance.clearTable();
    setState(() {
        _characters.clear();
    });
    if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Semua data berhasil
dihapus.')),
        );
    }
} catch (e) {
    if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('Terjadi kesalahan: $e')),
        );
    }
} finally {
    setState(() => _isLoading = false);
}

Future<void> _fetchFromAPI() async {
    setState(() => _isLoading = true);

    try {
        final response = await http.get(
            Uri.parse('https://jsonplaceholder.typicode.com/users'),
        );

        if (response.statusCode == 200) {
            final List<dynamic> dataJson = json.decode(response.body);
            await DatabaseHelper.instance.clearTable();

```

```

        List<Character> freshCharacters = [];

        for (var item in dataJson) {
            final char = Character.fromMap(item);
            freshCharacters.add(char);
            await DatabaseHelper.instance.insertCharacter(char);
        }

        setState(() => _characters = freshCharacters);

        if (mounted) {
            ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('Berhasil sync data dari
server!')),
            );
        }
    }
} catch (e) {
    if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Gagal konek internet,
pastikan online.')),
        );
    }
} finally {
    setState(() => _isLoading = false);
}
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(
            title: const Text('Ensiklopedia NPC'),
            actions: [
                IconButton(
                    icon: const Icon(Icons.delete_forever, color:
Colors.redAccent),
                    tooltip: 'Hapus Semua Data',
                    onPressed: _characters.isEmpty ? null : _deleteAllData,
                ),
                IconButton(

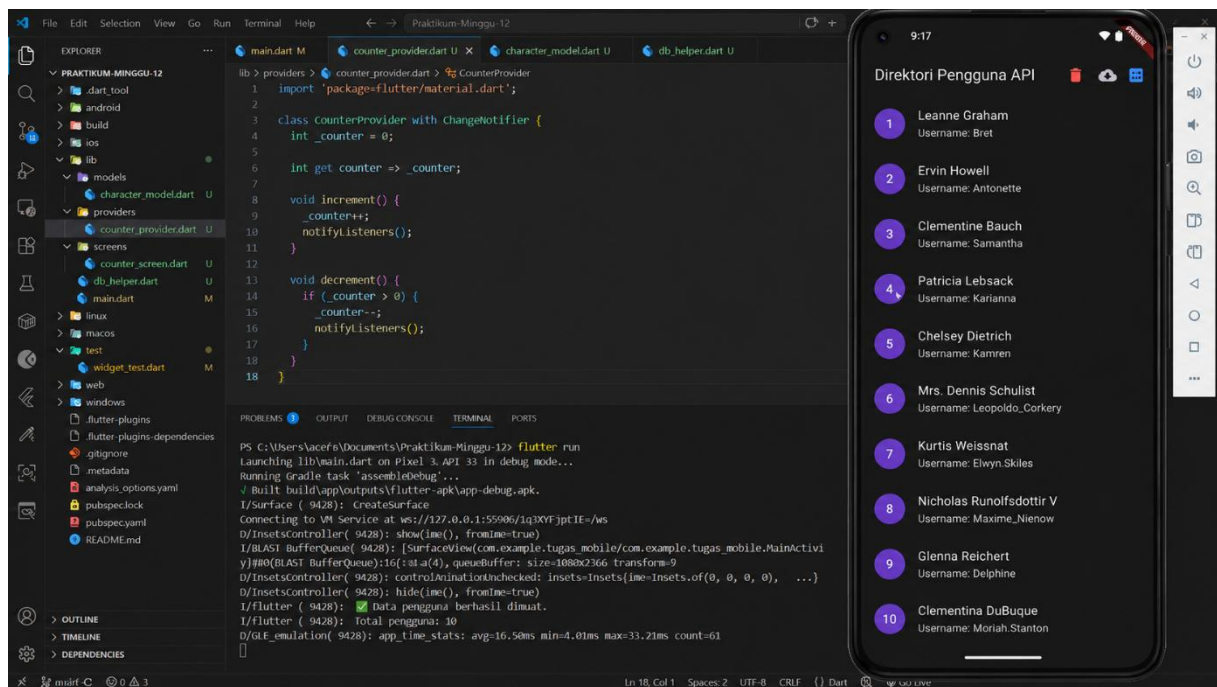
```

```

        icon: const Icon(Icons.cloud_download),
        tooltip: 'Sync dari Server',
        onPressed: _fetchFromAPI,
      ),
      IconButton(
        icon: const Icon(Icons.calculate, color:
Colors.blueAccent),
        tooltip: 'Buka Counter Provider',
        onPressed: () {
          Navigator.push(
            context,
            MaterialPageRoute(builder: (context) => const
CounterScreen()),
          );
        },
      ),
    ],
  ),
  body: _isLoading
    ? const Center(child: CircularProgressIndicator())
    : _characters.isEmpty
    ? const Center(child: Text('Data kosong.'))
    : ListView.builder(
        itemCount: _characters.length,
        itemBuilder: (context, index) {
          final char = _characters[index];
          return ListTile(
            leading: CircleAvatar(
              backgroundColor: Colors.deepPurple,
              child: Text(char.id.toString()),
            ),
            title: Text(char.name),
            subtitle: Text('Username: ${char.username}'),
          );
        },
      ),
);
}
}

```


ScreenShot



Deskripsi:

Gambar di atas menunjukkan tampilan halaman Counter Provider yang digunakan untuk menerapkan konsep manajemen state menggunakan Provider pada Flutter. Pada halaman ini terdapat nilai counter yang dapat berubah ketika pengguna menekan tombol tambah atau kurang. Perubahan nilai counter dikelola oleh class CounterProvider, kemudian perubahan tersebut dikirimkan ke antarmuka menggunakan notifyListeners() agar tampilan dapat diperbarui secara otomatis.

Penggunaan Provider membuat pengelolaan data menjadi lebih terstruktur karena logika perubahan nilai dipisahkan dari tampilan aplikasi. Dengan demikian, halaman counter dapat menampilkan perubahan nilai secara langsung tanpa perlu memuat ulang seluruh aplikasi. Implementasi ini menunjukkan bahwa Provider dapat digunakan untuk mengatur state sederhana pada aplikasi Flutter secara lebih efisien dan mudah dikembangkan.